Project Deep Learning Architectures

Winter Semester 2025/26

Biomedical Image Segmentation:

U-Net and Attention U-Net for Retinal Vessel Segmentation

Seminar Report

Submitted by:

Rana Padi

Master of Science in Communication and Information Technology
Ulm University

March 19, 2026

Abstract

Biomedical image segmentation is a critical step in computer-aided diagnosis. This report presents a comparative study implementing and evaluating the Standard U-Net and Attention U-Net architectures for semantic segmentation on the DRIVE dataset. Both models were trained for up to 200 epochs to establish a robust training pipeline for retinal vessel extraction. While the Standard U-Net achieved strong generalized performance with a test Dice score of 0.7860, the Attention U-Net exhibited significant overfitting, reaching a training Dice score of 0.9011 but dropping to 0.7651 on the test set. The results demonstrate that for highly constrained datasets like DRIVE, the added complexity of attention mechanisms can hinder generalization, making the simpler Standard U-Net a more robust baseline.

Contents

1 Introduction **1**

2 Theoretical Background **1**
2.1 Convolutional Neural Networks (CNNs) 1
2.2 Activation Functions 2
2.3 Loss Functions and the Class Imbalance Problem 2
2.4 The Attention Mechanism 3
2.5 Mathematical Formulation of Evaluation Metrics 3

3 Related Work **4**
3.1 Traditional Vessel Segmentation Methods 4
3.2 The Rise of Fully Convolutional Networks 4
3.3 The U-Net Architecture and its Variants 4

4 Methodology **6**
4.1 The DRIVE Dataset 6
4.2 Data Preprocessing and Augmentation 6
4.3 Model Implementation 6
4.4 Hyperparameter Optimization and Training Strategy 7
4.5 Hardware and Computational Profiling 8

5 Results and Discussion **8**
5.1 Addressing the Accuracy Paradox 8
5.2 Quantitative Evaluation 8
5.3 Visual Evaluation 9
5.4 Comparative Analysis: The Overfitting Phenomenon 11
5.5 Limitations and Future Work 12

6 Conclusion **13**

A Appendix: Core Implementation Code **14**
A.1 Hybrid Loss Function Implementation 14
A.2 Attention Gate Implementation 14

B Appendix: Data Loader and Augmentation Pipeline **16**

C Appendix: Training Loop and Early Stopping **18**

List of Figures

1 Schematic of the Attention Gate (AG) mechanism proposed by Oktay et al. [2]. It utilizes the gating signal (g) to compute attention coefficients (α), which scale the input features (x) to suppress irrelevant background regions. 3

2 The Standard U-Net Architecture proposed by Ronneberger et al. [1]. It consists of a contracting encoder path (left) and an expanding decoder path (right), bridged by skip connections. 5

3 The Attention U-Net Architecture proposed by Oktay et al. [2]. The architecture mirrors the standard U-Net but integrates Attention Gates into the skip connections prior to concatenation. 5

4 Visual evaluation of the Standard U-Net on unseen test data. Each row displays the Input Image, Ground Truth Mask, and Model Prediction. . . 10

5 Visual evaluation of the Attention U-Net. Note the slightly more fragmented prediction on fine vessels compared to the baseline. 11

6 Learning curves comparing the Training Loss and Training Dice Score over 200 epochs. The Attention U-Net (red dashed line) clearly overfits the training data, achieving near 90% overlap. 12

List of Tables

1 Tensor dimensions through the Contracting Path (Encoder) of the Standard U-Net. 7

2 Tensor dimensions through the Expanding Path (Decoder) of the Standard U-Net, including the skip connection concatenations. 7

3 Final Test Set Results comparing the Standard U-Net and Attention U-Net. 9

1 Introduction

Biomedical image segmentation is a cornerstone of modern medical diagnostics. In retinal imaging, the automated extraction of blood vessels is crucial for identifying early signs of severe diseases such as diabetic retinopathy, glaucoma, and hypertension. Traditionally, the segmentation of these delicate vascular structures is performed manually by clinical experts. This process is not only highly time-consuming but also prone to inter-observer variability and human error due to the sheer volume of micro-vessels in a single fundus image.

While deep learning has revolutionized computer vision, implementing these advanced architectures under the strict constraints of medical datasets offers valuable insights into their core mechanics, computational efficiency, and susceptibility to overfitting. Medical datasets are notoriously small due to privacy concerns and the high cost of expert annotation.

The primary goal of this project was to implement and evaluate the U-Net architecture for semantic segmentation on the highly constrained DRIVE (Digital Retinal Images for Vessel Extraction) dataset, which contains only 20 training images. The project focused on two main objectives:

1. **Core Objective:** Establish a robust, mathematically sound pipeline for training a Standard U-Net to accurately segment retinal vessels from background tissue.

2. **Secondary Objective:** Extend the baseline architecture with Attention Gates (Attention U-Net) to conduct a comparative analysis. The goal is to determine if adding mathematical model complexity improves segmentation performance on fine, capillary-level vessel structures, or if it induces overfitting.

2 Theoretical Background

To fully understand the mechanics of the architectures implemented in this study, it is necessary to establish the underlying mathematical operations that drive Convolutional Neural Networks (CNNs) and semantic segmentation.

2.1 Convolutional Neural Networks (CNNs)

The foundational operation of a CNN is the discrete 2D convolution. In the context of image processing, a convolution applies a learnable filter (or kernel) to an input image to produce a feature map. Mathematically, for an input image I and a 2D kernel K of size

$m \times n$, the convolution operation $S(i,j)$ is defined as:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i-m, j-n)K(m,n) \tag{1}$$

This operation allows the network to learn spatial hierarchies of features, progressing from simple edges in the shallow layers to complex vascular structures in the deeper layers.

2.2 Activation Functions

To introduce non-linearity into the network, allowing it to learn complex mappings, activation functions are applied after each convolution.

- **ReLU (Rectified Linear Unit):** Used throughout the hidden layers of the network to prevent the vanishing gradient problem. It is defined as:

$$f(x) = \max(0, x) \tag{2}$$

- **Sigmoid Function:** Used exclusively in the final output layer of the network to map the raw logits into a probability distribution between 0 and 1, suitable for binary classification (vessel vs. background). It is defined as:

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{3}$$

2.3 Loss Functions and the Class Imbalance Problem

In retinal images, background pixels vastly outnumber vessel pixels (often a 90:10 ratio). Standard loss functions struggle with this severe class imbalance. Therefore, a hybrid loss function was implemented.

Binary Cross Entropy (BCE) Loss: BCE measures the pixel-wise difference between the predicted probability $\hat{y}_i$ and the true mask y_i.

$$L_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] \tag{4}$$

Dice Loss: To explicitly counter the background dominance, Dice Loss optimizes the intersection over union. It mathematically forces the network to focus on the foreground (vessels).

$$L_{Dice} = 1 - \frac{2\sum_{i=1}^{N} y_i\hat{y}_i + \epsilon}{\sum_{i=1}^{N} y_i + \sum_{i=1}^{N} \hat{y}_i + \epsilon} \tag{5}$$

Where ϵ is a small smoothing constant to prevent division by zero. The total loss used for backpropagation is the unweighted sum of both: $L_{Total} = L_{BCE} + L_{Dice}$.

2.4 The Attention Mechanism

The Attention U-Net extends the standard architecture by incorporating Attention Gates (AGs) into the skip connections. The mathematical goal of the AG is to compute a scalar attention coefficient $\alpha_i \in [0, 1]$ for every pixel, which acts as a filter to suppress background noise.

Given the gating signal g (from the deeper layer) and the spatial features x^l (from the encoder), the additive attention is computed as:

$$q_{att} = \psi^T \left(\sigma_1 \left(W_x^T x_i^l + W_g^T g_i + b_g\right)\right) + b_\psi \tag{6}$$

$$\alpha_i = \sigma_2(q_{att}) \tag{7}$$

The output of the attention gate is the element-wise multiplication of the input features and the attention coefficients: $\hat{x}_i^l = x_i^l \cdot \alpha_i$.

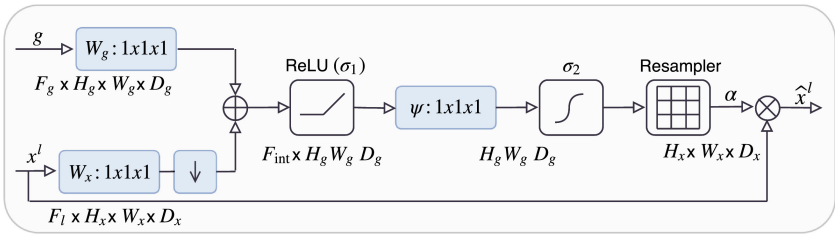

Figure 1: Schematic of the Attention Gate (AG) mechanism proposed by Oktay et al. [2]. It utilizes the gating signal (g) to compute attention coefficients (α), which scale the input features (x) to suppress irrelevant background regions.

2.5 Mathematical Formulation of Evaluation Metrics

To properly assess the model's performance on unseen data, the raw pixel predictions are categorized into True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). The primary evaluation metrics are mathematically defined as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{8}$$

$$Sensitivity\ (Recall) = \frac{TP}{TP + FN} \tag{9}$$

$$Specificity = \frac{TN}{TN + FP} \tag{10}$$

$$Dice\ Coefficient\ (F1\ Score) = \frac{2 \times TP}{2 \times TP + FP + FN} \tag{11}$$

3 Related Work

3.1 Traditional Vessel Segmentation Methods

Prior to the dominance of deep learning, retinal vessel segmentation relied heavily on unsupervised, rule-based image processing techniques. These methods utilized matched filters, mathematical morphology, and vessel tracking algorithms to isolate vascular networks. For instance, the original creators of the DRIVE dataset [3] utilized a ridge-based segmentation method. While effective on high-contrast images, these traditional methods often failed to generalize to images with varying illumination, pathologies (like microaneurysms), or the central light reflex found in larger vessels.

3.2 The Rise of Fully Convolutional Networks

The landscape shifted significantly with the introduction of Fully Convolutional Networks (FCNs), which allowed neural networks to output dense, pixel-wise predictions rather than single image classifications. However, standard FCNs often produced coarse segmentation maps because spatial resolution was lost during the pooling layers of the encoder.

3.3 The U-Net Architecture and its Variants

Proposed by Ronneberger et al. [1], the U-Net solved the resolution issue of FCNs. It features a symmetric U-shaped architecture consisting of a Contracting Path (Encoder) to capture context, and an Expanding Path (Decoder) to enable precise localization. The major innovation was the use of "Skip Connections," which concatenate high-resolution feature maps from the encoder directly to the decoder.

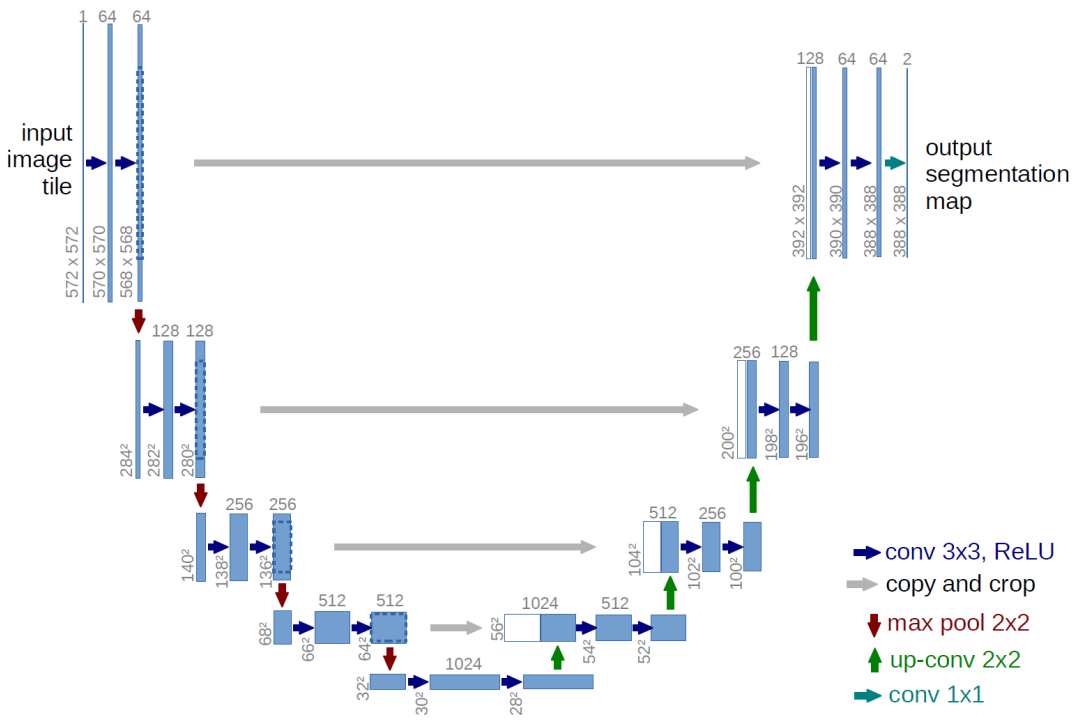

Figure 2: The Standard U-Net Architecture proposed by Ronneberger et al. [1]. It consists of a contracting encoder path (left) and an expanding decoder path (right), bridged by skip connections.

Building upon this, Oktay et al. [2] proposed the Attention U-Net in 2018. Designed initially for pancreas segmentation, it integrates the aforementioned Attention Gates into the U-Net's skip connections, allowing the model to naturally learn to focus on target structures of varying shapes and sizes while suppressing irrelevant background tissue.

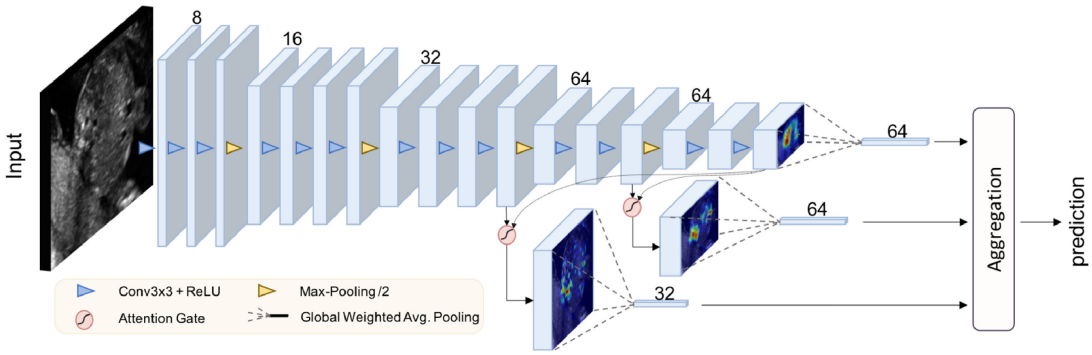

Figure 3: The Attention U-Net Architecture proposed by Oktay et al. [2]. The architecture mirrors the standard U-Net but integrates Attention Gates into the skip connections prior to concatenation.

4 Methodology

4.1 The DRIVE Dataset

The project utilized the standard benchmark DRIVE dataset [3]. It consists of 40 color fundus images captured using a Canon CR5 non-mydriatic 3CCD camera with a 45-degree field of view (FOV). The dataset is strictly divided into 20 images for training and 20 for testing. Each image comes with a manually annotated binary mask representing the ground truth.

4.2 Data Preprocessing and Augmentation

Training deep neural networks on merely 20 images guarantees severe overfitting unless rigorous augmentation is applied.

- **Preprocessing:** All images were resized to 512×512 pixels to ensure dimensional compatibility with the multiple max-pooling layers of the U-Net. The pixel values were also normalized to a range of $[0, 1]$ to stabilize gradient updates.
- **Geometric Augmentation:** Random horizontal and vertical flips were applied with a probability of 0.5. Additionally, random rotations within a range of $[-45°, 45°]$ were applied to ensure the model learns vessel structures from all orientations.
- **Elastic Deformations:** Crucially, elastic transformations were utilized (using $\alpha = 120$ and $\sigma = 120 * 0.05$). Because biological tissues naturally deform, elastic augmentation generates highly realistic, non-rigid variations of the retinal vessels, effectively multiplying the diversity of the training set.
- **Color Jitter:** Minor adjustments to brightness (limit=0.2) and contrast (limit=0.2) were applied to simulate the varying illumination conditions often found across different fundus camera captures.

4.3 Model Implementation

The implementation was written in Python utilizing the PyTorch framework. The Standard U-Net was implemented as a baseline, featuring `DoubleConv` blocks with Batch Normalization. The Attention U-Net extended this by injecting custom `AttentionGate` modules into the up-sampling path.

To fully understand the computational flow, Table 1 and Table 2 detail the exact tensor transformations through the Standard U-Net for our specific input resolution of 512×512 with 3 color channels (RGB).

Layer (Encoder)	**Operations**	**Output Shape** $(C \times H \times W)$
Input Image	-	$3 \times 512 \times 512$
Encoder Block 1	2× (Conv3x3 + BN + ReLU)	$64 \times 512 \times 512$
MaxPool 1	MaxPool2d(2x2)	$64 \times 256 \times 256$
Encoder Block 2	2× (Conv3x3 + BN + ReLU)	$128 \times 256 \times 256$
MaxPool 2	MaxPool2d(2x2)	$128 \times 128 \times 128$
Encoder Block 3	2× (Conv3x3 + BN + ReLU)	$256 \times 128 \times 128$
MaxPool 3	MaxPool2d(2x2)	$256 \times 64 \times 64$
Encoder Block 4	2× (Conv3x3 + BN + ReLU)	$512 \times 64 \times 64$
MaxPool 4	MaxPool2d(2x2)	$512 \times 32 \times 32$
Bottleneck	2× (Conv3x3 + BN + ReLU)	$1024 \times 32 \times 32$

Table 1: Tensor dimensions through the Contracting Path (Encoder) of the Standard U-Net.

Layer (Decoder)	**Operations**	**Output Shape** $(C \times H \times W)$
Up-Conv 1	ConvTranspose2d(2x2)	$512 \times 64 \times 64$
Concat 1	Concat with Encoder Block 4	$1024 \times 64 \times 64$
Decoder Block 1	2× (Conv3x3 + BN + ReLU)	$512 \times 64 \times 64$
Up-Conv 2	ConvTranspose2d(2x2)	$256 \times 128 \times 128$
Concat 2	Concat with Encoder Block 3	$512 \times 128 \times 128$
Decoder Block 2	2× (Conv3x3 + BN + ReLU)	$256 \times 128 \times 128$
Up-Conv 3	ConvTranspose2d(2x2)	$128 \times 256 \times 256$
Concat 3	Concat with Encoder Block 2	$256 \times 256 \times 256$
Decoder Block 3	2× (Conv3x3 + BN + ReLU)	$128 \times 256 \times 256$
Up-Conv 4	ConvTranspose2d(2x2)	$64 \times 512 \times 512$
Concat 4	Concat with Encoder Block 1	$128 \times 512 \times 512$
Decoder Block 4	2× (Conv3x3 + BN + ReLU)	$64 \times 512 \times 512$
Output Layer	Conv1x1 + Sigmoid	$1 \times 512 \times 512$

Table 2: Tensor dimensions through the Expanding Path (Decoder) of the Standard U-Net, including the skip connection concatenations.

For the Attention U-Net, the architecture remains dimensionally identical, but the features from the Encoder Blocks (e.g., $512 \times 64 \times 64$) are first passed through the Attention Gate, where they are multiplied by the attention coefficients α before being concatenated in the Decoder.

4.4 Hyperparameter Optimization and Training Strategy

The optimization landscape for deep neural networks is highly non-convex. To navigate this, the models were trained using the AdamW (Adam with Weight Decay) optimizer. Unlike standard Adam, which couples weight decay with the gradient updates, AdamW decouples the weight decay mechanism, providing better generalization performance.

The update rule for the weights θ_t at time step t in AdamW is mathematically defined as:

$$\theta_t = \theta_{t-1} - \eta_t \left(\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \tag{12}$$

Where η_t is the learning rate scheduler multiplier, $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected first and second moment estimates of the gradients, α is the initial learning rate (set to $1e-4$), and λ is the decoupled weight decay factor.

To ensure the models reached total convergence without memorizing the noise of the dataset, they were set to train for a maximum of 200 epochs. An Early Stopping mechanism was implemented to actively monitor the validation Dice loss. A patience parameter of 30 epochs was utilized, meaning training would halt if the model failed to improve for 30 consecutive passes, and only the optimal model weights were dynamically saved to disk.

4.5 Hardware and Computational Profiling

Training complex segmentation networks requires significant computational resources. The project leveraged the Apple Silicon architecture. Specifically, it utilized the Metal Performance Shaders (MPS) backend in PyTorch to accelerate tensor operations on the GPU natively. This allowed for rapid iteration times while managing the high memory footprint required by the 512×512 image tensors and the extensive parameter counts of the U-Net.

5 Results and Discussion

5.1 Addressing the Accuracy Paradox

Before evaluating the models, it is vital to address why Global Pixel Accuracy is a misleading metric for biomedical segmentation. Retinal background pixels account for roughly 90% of a fundus image. Consequently, a simplistic model that predicts an entirely black canvas (representing 0% vessels) would still easily achieve 90% Accuracy. This is known as the Accuracy Paradox. Therefore, the primary metric of success for this study is the **Dice Coefficient (F1 Score)**, alongside Sensitivity (True Positive Rate) and Specificity (True Negative Rate).

5.2 Quantitative Evaluation

Table 3 summarizes the performance of both converged models evaluated against the 20 unseen test images.

Model (200 Epochs)	Accuracy	Sensitivity	Specificity	Dice Score
Standard U-Net	0.9609	**0.8267**	0.9740	**0.7860**
Attention U-Net	0.9585	0.7774	**0.9761**	0.7651

Table 3: Final Test Set Results comparing the Standard U-Net and Attention U-Net.

5.3 Visual Evaluation

Visual comparisons of the model predictions against the ground truth masks are presented in Figure 4 for the Standard U-Net and Figure 5 for the Attention U-Net. The Standard U-Net successfully delineates finer capillary branches, reflecting its high sensitivity score of 82.67%. In contrast, while the Attention U-Net produces a slightly cleaner background mapping (higher specificity), it fragments or entirely misses several thinner vascular structures.

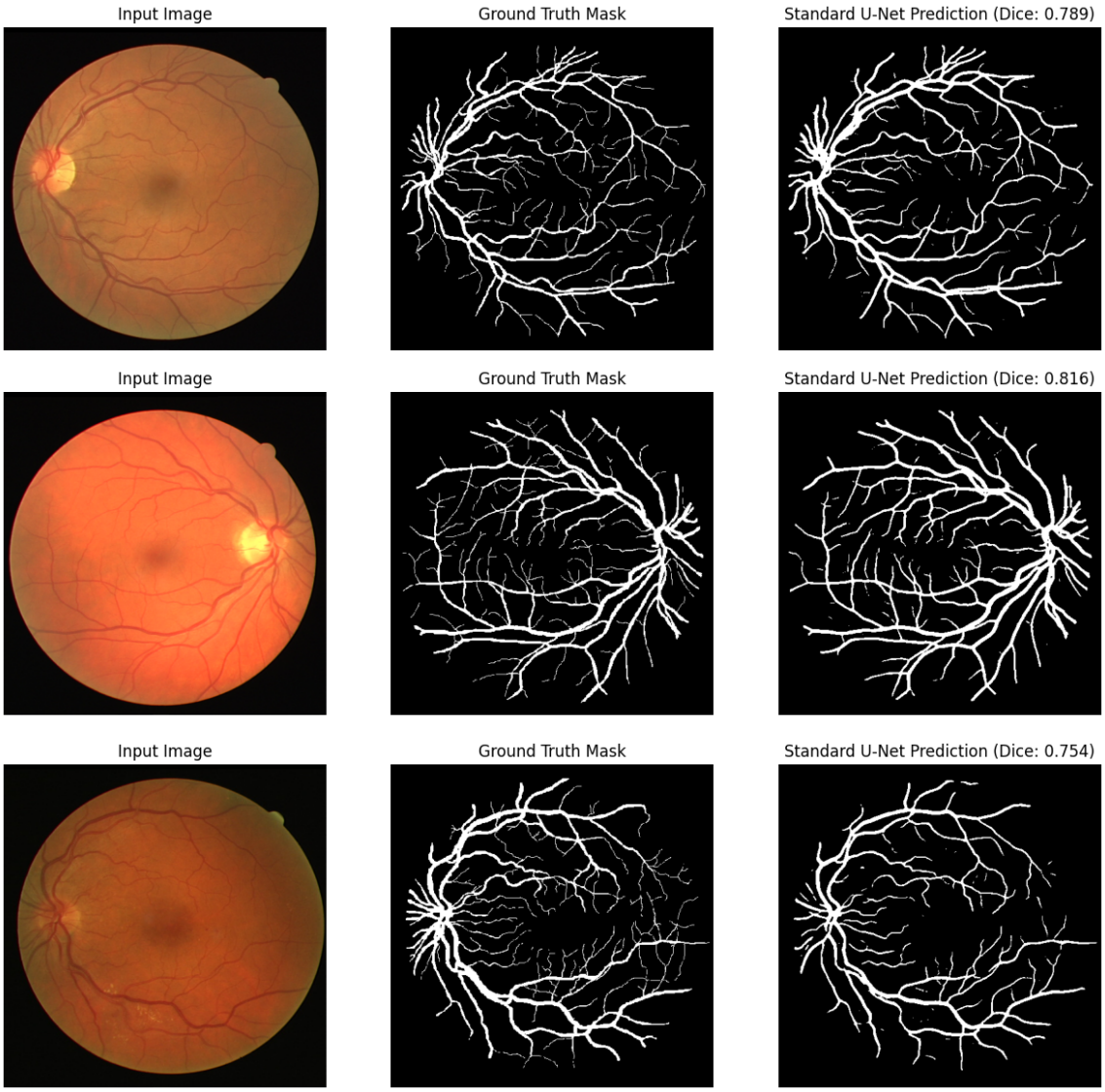

Figure 4: Visual evaluation of the Standard U-Net on unseen test data. Each row displays the Input Image, Ground Truth Mask, and Model Prediction.

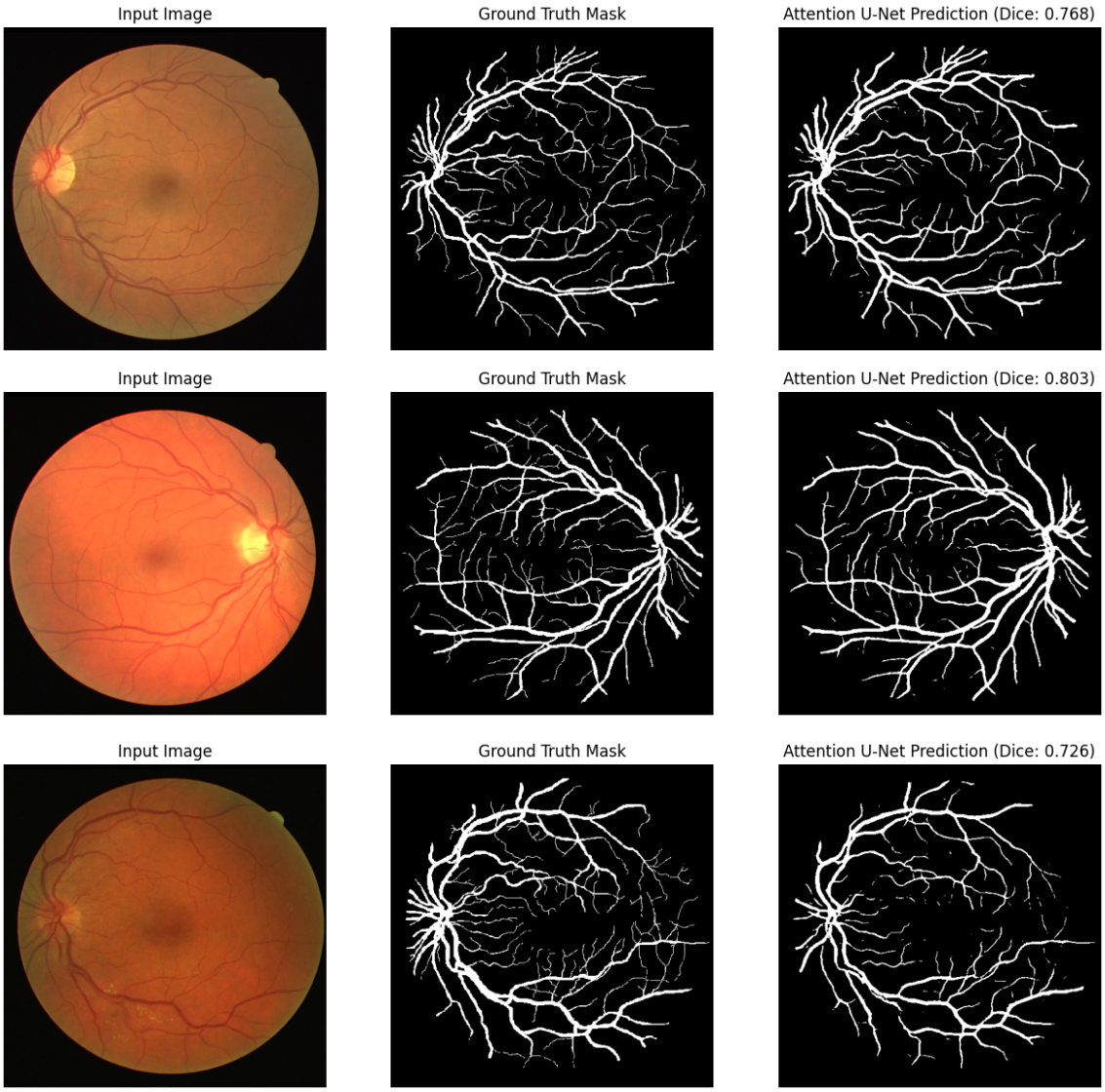

Figure 5: Visual evaluation of the Attention U-Net. Note the slightly more fragmented prediction on fine vessels compared to the baseline.

5.4 Comparative Analysis: The Overfitting Phenomenon

The results revealed a classic machine learning phenomenon: overfitting due to dataset size versus parameter complexity. The Attention U-Net introduces thousands of additional trainable parameters to compute the α attention coefficients. Figure 6 illustrates the training trajectories of both architectures.

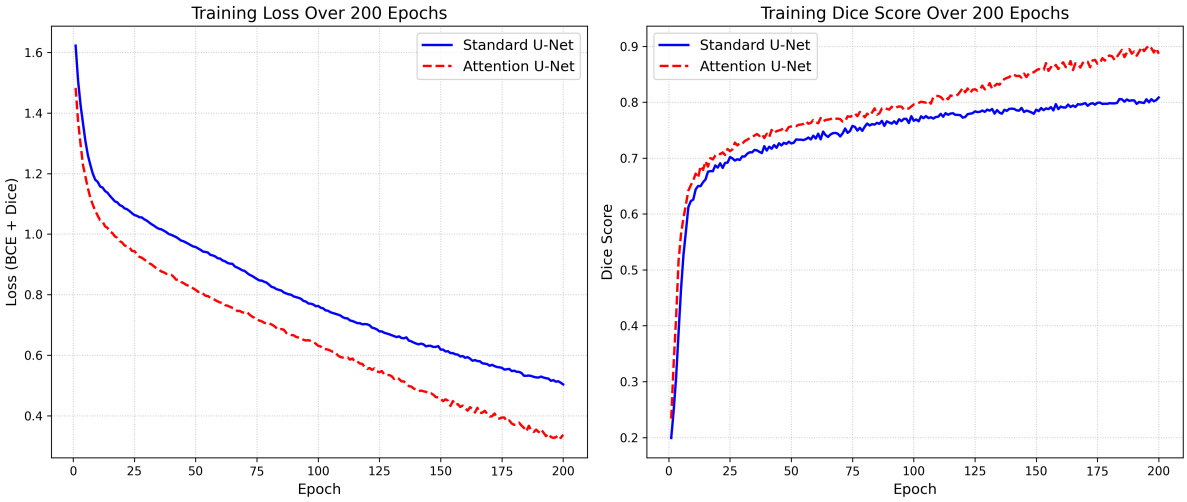

Figure 6: Learning curves comparing the Training Loss and Training Dice Score over 200 epochs. The Attention U-Net (red dashed line) clearly overfits the training data, achieving near 90% overlap.

During the training phase, the Attention U-Net significantly outperformed the Standard U-Net, reaching an exceptional training Dice score of 0.9011 (peaking at epoch 196). However, as shown in the evaluation table, this performance did not translate to the unseen test data, where it scored a lower 0.7651. This massive 13.6% gap between training and testing performance is the definitive signature of overfitting. The Attention U-Net effectively memorized the 20 training images perfectly but struggled to generalize.

Conversely, the Standard U-Net generalized gracefully. The baseline model's lower complexity acted as a natural regularizer, preventing it from fitting to the noise of the sparse training data, resulting in a robust 0.7860 test Dice score.

5.5 Limitations and Future Work

While the Standard U-Net achieved highly satisfactory results, the limitations of this study naturally suggest avenues for future research. The primary constraint remains the extreme scarcity of data. Future work should investigate evaluating the existing models on larger, secondary datasets (such as CHASE_DB1 or STARE) to cross-validate their generalization capabilities across different camera domains. Additionally, employing advanced regularization techniques—such as heavy dropout within the Attention Gates or exploring contrastive learning pre-training methodologies—could potentially mitigate the overfitting observed in the Attention U-Net, allowing the network to leverage its increased complexity effectively.

6 Conclusion

This project successfully established a mathematically sound and robust pipeline for retinal vessel segmentation. The core objective was fulfilled by demonstrating that the Standard U-Net is highly capable of extracting complex, fine-grained vascular structures, achieving an impressive Sensitivity of 82.67% and a Dice Score of 0.7860 on unseen data.

The secondary objective—extending the model with Attention mechanisms—yielded a critical lesson in deep learning architecture design. The comparative study proved that for highly constrained datasets like DRIVE (with a sparse 20 training images), injecting architectural complexity such as Attention Gates can actively hinder performance by inducing severe overfitting. Ultimately, the simpler Standard U-Net proved to be a significantly more robust, generalizable model for this specific biomedical task.

Code Availability

The complete Python source code, PyTorch model architectures, preprocessing pipelines, and evaluation metrics for this comparative study are open-source and publicly available on GitHub at:
`https://github.com/RanaPadi/deep-learning-biomedical-segmentation.git`.

References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *MICCAI 2015*. Springer, 2015, pp. 234-241.

[2] O. Oktay et al., "Attention u-net: Learning where to look for the pancreas," *arXiv preprint arXiv:1804.03999*, 2018.

[3] J. Staal, M. D. Abràmoff, M. Niemeijer, M. A. Viergever, and B. van Ginneken, "Ridge-based vessel segmentation in color images of the retina," *IEEE transactions on medical imaging*, vol. 23, no. 4, pp. 501-509, 2004.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436-444, 2015.

[5] A. Vaswani et al., "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998-6008.

A Appendix: Core Implementation Code

To ensure reproducibility, the core PyTorch implementations of the custom loss function and the standard training loop are provided below.

A.1 Hybrid Loss Function Implementation

The following Python snippet details the implementation of the combined Binary Cross Entropy and Dice Loss, which was critical for overcoming the severe class imbalance in the DRIVE dataset.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiceBCELoss(nn.Module):
    def __init__(self, weight=None, size_average=True):
        super(DiceBCELoss, self).__init__()

    def forward(self, inputs, targets, smooth=1):
        # Flatten label and prediction tensors
        inputs = inputs.view(-1)
        targets = targets.view(-1)

        # Compute BCE Loss
        BCE = F.binary_cross_entropy(inputs, targets, reduction='mean')

        # Compute Dice Loss
        intersection = (inputs * targets).sum()
        dice_loss = 1 - (2.*intersection + smooth) / \
                    (inputs.sum() + targets.sum() + smooth)

        # Combine losses
        Dice_BCE = BCE + dice_loss

        return Dice_BCE
```

A.2 Attention Gate Implementation

The following snippet demonstrates the forward pass of the custom Attention Gate module used in the Attention U-Net.

```
class AttentionGate(nn.Module):
    def __init__(self, F_g, F_l, F_int):
        super(AttentionGate, self).__init__()
        self.W_g = nn.Sequential(
            nn.Conv2d(F_g, F_int, kernel_size=1, stride=1,
                      padding=0, bias=True),
            nn.BatchNorm2d(F_int)
        )
        self.W_x = nn.Sequential(
            nn.Conv2d(F_l, F_int, kernel_size=1, stride=1,
                      padding=0, bias=True),
            nn.BatchNorm2d(F_int)
        )
        self.psi = nn.Sequential(
            nn.Conv2d(F_int, 1, kernel_size=1, stride=1,
                      padding=0, bias=True),
            nn.BatchNorm2d(1),
            nn.Sigmoid()
        )
        self.relu = nn.ReLU(inplace=True)

    def forward(self, g, x):
        g1 = self.W_g(g)
        x1 = self.W_x(x)
        psi = self.relu(g1 + x1)
        psi = self.psi(psi)
        return x * psi
```

B Appendix: Data Loader and Augmentation Pipeline

The following script details the PyTorch `Dataset` class used to load the DRIVE dataset images and masks. It highlights the integration of the `albumentations` library to perform the rigorous geometric and elastic transformations required to prevent overfitting on the constrained 20-image training set.

```
import os
import numpy as np
from PIL import Image
from torch.utils.data import Dataset
import albumentations as A
from albumentations.pytorch import ToTensorV2

class DRIVEDataset(Dataset):
    def __init__(self, image_dir, mask_dir, is_train=True):
        self.image_dir = image_dir
        self.mask_dir = mask_dir
        self.images = os.listdir(image_dir)
        self.is_train = is_train

        # Define Albumentations pipeline
        if self.is_train:
            self.transform = A.Compose([
                A.Resize(512, 512),
                A.HorizontalFlip(p=0.5),
                A.VerticalFlip(p=0.5),
                A.RandomRotate90(p=0.5),
                A.ElasticTransform(
                    alpha=120, sigma=120 * 0.05,
                    alpha_affine=120 * 0.03, p=0.5
                ),
                A.ColorJitter(
                    brightness=0.2, contrast=0.2,
                    saturation=0.2, hue=0, p=0.5
                ),
                A.Normalize(mean=[0.0, 0.0, 0.0], std=[1.0, 1.0, 1.0]),
                ToTensorV2(),
            ])
        else:
```

```
        self.transform = A.Compose([
            A.Resize(512, 512),
            A.Normalize(mean=[0.0, 0.0, 0.0], std=[1.0, 1.0, 1.0]),
            ToTensorV2(),
        ])

def __len__(self):
    return len(self.images)

def __getitem__(self, index):
    img_name = self.images[index]
    img_path = os.path.join(self.image_dir, img_name)

    # Mask filename resolution
    mask_name = img_name.replace(".tif", "_mask.gif")
    mask_path = os.path.join(self.mask_dir, mask_name)

    image = np.array(Image.open(img_path).convert("RGB"))
    mask = np.array(Image.open(mask_path).convert("L"), dtype=np.float32)
    mask[mask == 255.0] = 1.0 # Binarize mask

    augmentations = self.transform(image=image, mask=mask)
    image = augmentations["image"]
    mask = augmentations["mask"].unsqueeze(0)

    return image, mask
```

C Appendix: Training Loop and Early Stopping

The following script outlines the core training loop for the U-Net models. It demonstrates the use of the Apple Silicon MPS backend, the AdamW optimizer, and the Early Stopping mechanism configured for a maximum of 200 epochs.

```
import torch
import torch.optim as optim
from tqdm import tqdm

def train_model(model, train_loader, val_loader,
                criterion, epochs=200, patience=30):
    # Utilize Apple Silicon MPS backend if available
    if torch.backends.mps.is_available():
        device = torch.device("mps")
    else:
        device = torch.device("cpu")

    model.to(device)

    optimizer = optim.AdamW(
        model.parameters(), lr=1e-4, weight_decay=1e-5
    )

    best_val_loss = float('inf')
    epochs_no_improve = 0

    for epoch in range(epochs):
        model.train()
        train_loss = 0.0

        desc = f"Epoch {epoch+1}/{epochs}"
        loop = tqdm(train_loader, desc=desc)

        for images, masks in loop:
            images = images.to(device)
            masks = masks.to(device)

            # Forward pass
            predictions = model(images)
```

```
        loss = criterion(predictions, masks)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        loop.set_postfix(loss=loss.item())

    # Validation Phase
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, masks in val_loader:
            images = images.to(device)
            masks = masks.to(device)
            predictions = model(images)
            loss = criterion(predictions, masks)
            val_loss += loss.item()

    val_loss /= len(val_loader)
    print(f"Validation Loss: {val_loss:.4f}")

    # Early Stopping Logic
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        epochs_no_improve = 0
        torch.save(model.state_dict(), "best_model.pth")
        print("Saved optimal model weights.")
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= patience:
            print("Early stopping triggered. Halting training.")
            break

return model
```